

Keywords: state space models • long-sequence modeling • hierarchical memory • recurrent neural networks

Mamba with Hierarchical Memory: Solving the Representation Bottleneck in Long Sequence Modeling

HMM Adds Paragraph-Level Semantics and Persistent Long-Term Memory to Mamba’s Fixed Recurrent State, Enabling Near-Linear Long-Context Reasoning

By Qinwen Wang, Jieping Luo, Aoxiang Qin, Ruoyu Zhao, Jianxiong Tang, Wei Zhang, Zhichao Lu, Luziwei Leng

arXiv:2608.02347 • August 2026

State space models like Mamba offer linear-time sequence processing, but their fixed-capacity recurrent state is a fundamental bottleneck on long sequences: old context is progressively overwritten as the sequence grows. Hierarchical Memory Mamba (HMM) resolves this by adding two lightweight memory tiers above the base recurrent state—paragraph-level semantic extraction and persistent long-term memory retrieval—enabling near-linear complexity long-range reasoning on sequences where Mamba’s performance degrades.

AT A GLANCE

Problem: Mamba’s fixed-size recurrent state overwrites old context as sequences grow, creating a representation bottleneck that limits long-range recall.

Why it matters: Long-document QA, many-shot ICL, and genomic sequence modeling require retaining information across scales far beyond the recurrent state capacity.

Method: Hierarchical Memory Mamba (HMM) extracts paragraph-level semantics from the hidden state and stores them in a persistent long-term memory for task-relevant retrieval.

Result: Large improvements on Long-Range Arena, SCROLLS benchmarks, and many-shot ICL tasks; near-linear complexity vs. quadratic for Transformers.

Evidence: Evaluated on six Long-Range Arena tasks, four SCROLLS benchmarks, and PG-19 language modeling up to 32k tokens.

The Big Picture

Recurrent sequence models like Mamba process tokens in $O(N)$ time and $O(1)$ memory per step, making them far cheaper than Transformers for long sequences. But

the constant memory comes with a hard limit: the recurrent hidden state $h_t \in \mathbb{R}^d$ must compress *all* context seen so far into a fixed-size vector at every step. On short sequences, this works well. On sequences of thousands or tens of thousands of tokens, earlier context is progressively overwritten—a phenomenon the authors call the *representation bottleneck*.

The problem is architectural, not a matter of training. No amount of fine-tuning on long sequences fixes the fundamental capacity limit of a fixed recurrent state. HMM attacks the bottleneck directly by introducing a structured memory hierarchy above the recurrent state, inspired by the cognitive science of human memory (sensory $\rightarrow$ working $\rightarrow$ long-term memory).

Why Existing Approaches Fall Short

Larger recurrent state. Expanding d reduces the bottleneck but increases per-step compute quadratically and does not scale gracefully.

Mamba-2 / HGRN2. These architectural improvements to SSMs increase state capacity and improve practical long-context performance, but the fixed-state bottleneck remains—performance still degrades on Path-X and other tasks requiring recall beyond the effective state horizon.

Hybrid Mamba-Transformer models (Jamba, Zamba) insert full attention layers at regular intervals, recovering long-range recall at the cost of $O(N^2)$ for those layers. HMM achieves near-linear complexity throughout by using sparse key-value retrieval rather than full attention for the long-term memory.

KV-cache SSMs. Some recent models cache a rolling window of past hidden states and attend over them (e.g. RWKV-7’s chunk attention). Window-limited KV caches still impose a horizon; HMM’s long-term memory has no fixed horizon.

Core Mechanism

HMM adds two memory tiers to a standard Mamba backbone:

Tier 1—Sensory Memory (Mamba hidden state). The standard recurrent state h_t processes the current token in the context of recent tokens. It is fast and fine-grained but has fixed capacity d .

Tier 2—Working Memory (PLS extractor). Every P tokens (a “paragraph”), a lightweight cross-attention module reads the P hidden states $h_{[kP:(k+1)P]}$ and compresses them into a single *Paragraph-Level Semantic* (PLS) vector:

$$\text{PLS}_k = \text{MultiHead}(Q_{\text{probe}}, K = W_K H_k, V = W_V H_k),$$
 where $H_k = [h_{kP}, \dots, h_{(k+1)P-1}] \in \mathbb{R}^{P \times d}$ and $Q_{\text{probe}} \in$

$\mathbb{R}^{q \times d}$ is a learned query matrix ($q \ll P$). The PLS captures the slow, paragraph-scale semantics that the fast sensory memory cannot retain.

Tier 3—Long-Term Memory (LTM store). PLS vectors accumulate in a growing bank $\mathcal{M} = \{(\text{PLS}_k, k)\}$. At each step, the current hidden state queries the bank via k -NN retrieval followed by a small attention:

$$\text{retrieved}_t = \text{Attn}(W_q h_t, K_{\text{LTM}}, V_{\text{LTM}}),$$

and the retrieved content is injected into the Mamba state update via a gating mechanism:

$$h'_t = h_t + \sigma(W_g h_t) \odot \text{retrieved}_t.$$

Technical Formulation

The complexity of HMM for a sequence of N tokens with paragraph size P :

Sensory processing: $O(Nd)$ (linear, standard Mamba).

PLS extraction: One cross-attention over P tokens per paragraph: $O((N/P) \cdot P \cdot q \cdot d) = O(Nqd/1)$, effectively $O(N)$ for fixed q .

LTM retrieval: N/P entries in the bank; each retrieval is $O(N/P)$ for exact attention over all entries, giving $O(N \cdot N/P) = O(N^2/P)$ total. For $P \gg 1$ (e.g. $P = 256$), this is orders of magnitude cheaper than full attention ($O(N^2)$).

The training objective adds no new loss terms: HMM is trained end-to-end on the next-token prediction objective, with the PLS extractor and LTM read/write operations differentiable throughout.

Learning or Inference Procedure

Initialization. HMM can be initialized from a pretrained Mamba checkpoint; only the PLS extractor and LTM projection matrices are new parameters (typically $<5\%$ of total parameters).

Fine-tuning. Continued pre-training on long-document corpora for 10–20k steps suffices to activate the LTM pathway; full long-context fine-tuning with HMM from scratch is also supported.

Inference. The LTM bank grows as the sequence is processed; retrieval uses approximate nearest neighbors (FAISS) for bank sizes above 10k entries. The k -NN search cost is sub-linear in practice due to FAISS’s IVF indexing.

What the Guarantee Says

Monotone recall. A PLS vector is written for every paragraph in the input without overwriting; the LTM bank is append-only. This guarantees that information from paragraph k is always retrievable from step $k' > k$, unlike the sensory state which is overwritten at step $k + 1$.

Complexity bound. For paragraph size $P = O(\sqrt{N})$, LTM retrieval cost is $O(N^{3/2})$ —between linear and quadratic. For fixed P , it is $O(N^2/P)$, which for $P = 256$ and $N = 32,768$ is $128\times$ cheaper than full attention.

Experimental Findings

- **Long-Range Arena:** HMM outperforms Mamba-2 by 4–9 points on Path-X and Retrieval tasks, where long-range recall is decisive.
- **SCROLLS (QuALITY):** 6-point accuracy improvement over Mamba-2 on multi-hop reading comprehension requiring cross-paragraph reasoning.
- **Many-shot ICL:** At 64-shot, HMM accuracy is 3 points above Mamba-2 and continues improving; Mamba-2 saturates and slightly degrades.
- **PG-19 language modeling:** 0.4 perplexity reduction at 16k tokens; 0.9 reduction at 32k tokens, showing increasing benefit at longer contexts.
- **Parameter overhead:** $<5\%$ additional parameters for the PLS extractor and LTM projections.

Ablations and Interpretation

Paragraph size P . Smaller P gives richer PLS resolution but increases LTM bank growth rate and retrieval cost. $P = 128$ – 256 gives the best quality/cost tradeoff in experiments.

LTM vs. no LTM. Removing the LTM (keeping only PLS extraction) gives intermediate improvement over base Mamba but still degrades on the longest sequences, confirming that persistent retrieval—not just better compression—is needed.

Retrieval head count. Using 4 retrieval attention heads vs. 1 improves SCROLLS accuracy by 2 points; beyond 4 heads, gains are marginal.

Reference

Qinwen Wang, Jieping Luo, Aoxiang Qin, Ruoyu Zhao, Jianxiong Tang, Wei Zhang, Zhichao Lu, Luziwei Leng. **Mamba with Hierarchical Memory: Solving Representation Bottleneck in Long Sequence Modeling.** arXiv:2608.02347, August 2026. <https://arxiv.org/abs/2608.02347>